Davante

16

ENTORNOS DE DESARROLLO

Pruebas de código

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Cubrimiento, valores límite y clases de equivalencia 4

/ 3. Pruebas unitarias de clases y funciones 4

/ 4. Caso práctico 1: “Uso de valores límites” 5

/ 5. Automatización de pruebas 6

/ 6. Calidad del software 6

/ 7. Caso práctico 2: “Validación de datos” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Comprender el concepto de calidad software.

Conocer diferentes herramientas para pruebas automáticas.

Saber aplicar diferentes tipos de pruebas.

Conocer el procedimiento para las pruebas software.

/ 1. Introducción y contextualización práctica

Como usuarios de aplicaciones elegimos aquellas que nos ofrecen unos ciertos estándares de calidad, que no provoquen que podamos perder el trabajo que hayamos realizado, que no expongan nuestros datos, etc.

Por estas razones, desde el punto de vista del desarrollador de software es necesario que tengamos en cuenta los diferentes aspectos que pueden influir en la calidad del producto que generemos.

Existen diferentes medidas que permiten garantizar la calidad de un producto software, entre otras, los estándares de calidad que se pueden aplicar a diferentes productos y contextos, no solo en el desarrollo software en sí.

Uno de los aspectos que solemos olvidar cuando hablamos de la calidad, es la fiabilidad de una aplicación. Gracias a la fiabilidad podemos estar seguros de que podemos utilizar un producto en diferentes contextos y que no fallará.

La seguridad es otro de los aspectos que se suele descuidar. En este caso solo nos acordamos de la seguridad cuando hay un problema grave, en caso contrario podemos ser muy laxos. Sin embargo, la seguridad tiene que ver directamente con la calidad y fiabilidad de un producto.

En este tema destacaremos diferentes formas de analizar la calidad del software y realizar las pruebas necesarias para lograrlo.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.



Fig. 1. Las herramientas adecuadas permiten aumentar la calidad.

Audio Intro. "Incumplimiento de normas ISO"
http://bit.ly/3pYwrPO

/ 2. Cubrimiento, valores límite y clases de equivalencia

Desde un punto de vista ideal, las pruebas del código fuente deberían ser capaces de probar todas las posibles alternativas y combinaciones existentes. Si no tenemos en cuenta el contexto de ejecución, esto podría ser posible, aunque las combinaciones fuesen muy elevadas.

Sin embargo, cuando consideramos el contexto de ejecución existen casos que pueden escapar a nuestro control. Por ello, es necesario establecer el **nivel de concreción que queremos en las pruebas que vayamos a realizar.** En ocasiones, las pruebas del código van orientadas en diferentes sentidos que mostramos a continuación:

- **Pruebas de cobertura o cubrimiento.** Este tipo de pruebas es relativamente sencillo de llevar a cabo, pues la idea es **poder ejecutar al menos una vez cada línea de código.** En muchos entornos de desarrollo se nos muestran fácilmente cuántas veces se ejecuta una determinada línea y podríamos contarlo incluso manualmente. Sin embargo, ya existen herramientas que permiten realizar esto automáticamente **y ver si el código ha pasado por ahí durante su ejecución.**

- **Pruebas de casos extremos (valores límite).** En ocasiones, el código que desarrollamos se comporta correctamente, pero podemos distinguir que existen **casos excepcionales en los que el comportamiento debería ser tratado de una forma diferente.** Por ejemplo, en el caso de la división podemos ver que un caso extremo sería la división por cero.

  Como podemos ver, para este ejemplo este tipo de división sería un caso particular pues tendríamos que informar de una entrada no válida.

- **Pruebas de clases de equivalencia.** Este tipo de pruebas provienen de un **concepto matemático que son las funciones de equivalencia.** A través de estas funciones se pueden ver que no es necesario **comprobar** todas **las entradas de una función** sino simplemente **valores equivalentes.** De esta forma los casos de prueba se reducen significativamente.

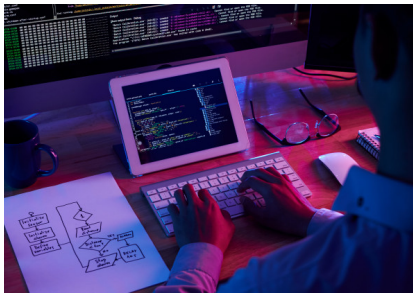


Fig. 2. Pruebas de código.

Audio 1. "Definición de casos de prueba"
http://bit.ly/39cnCLl

/ 3. Pruebas unitarias de clases y funciones

Las pruebas unitarias son aquellas en las que **podemos probar los componentes de manera individual.** Se sitúan por debajo de las pruebas de integración, siendo éstas las siguientes en poder ejecutarse.

El objetivo de las pruebas unitarias es **poder validar cada porción del *software*** para comprobar si cumplen aquello para lo que fue diseñado. Normalmente, las pruebas unitarias prueban una porción relativamente pequeña del código, que puede incluir funciones, procedimientos, etc.

Teóricamente, una prueba unitaria se centra en comprobar todos los posibles caminos que puede tener una determinada funcionalidad. La idea es que podamos ser capaces de cubrir todo el código, con sus posibles alternativas, a través de funcionalidades o pruebas simples. Sin embargo, en la práctica puede ser muy complicado crear pruebas unitarias para probar únicamente una porción del código.

Existen **herramientas para crear pruebas unitarias de manera sencilla:**

- **JUnit.** Se trata de un conjunto de clases cuya principal misión es poder ejecutar pruebas unitarias de una manera segura en el lenguaje Java. Actualmente este framework se utiliza en diferentes entornos para facilitar el diseño de las pruebas unitarias. Por ejemplo, es utilizado en NetBeans, Eclipse, Android Studio, etc.

  A través de los plugins es posible añadir esta funcionalidad a diferentes IDEs. Además, incluye funcionalidades para poder gestionar las pruebas de regresión.

- **PHPUnit.** Se trata del equivalente a JUnit para realizar pruebas en PHP.

- **NUnit.** En este caso se ofrece la gestión de pruebas unitarias para entornos .NET.

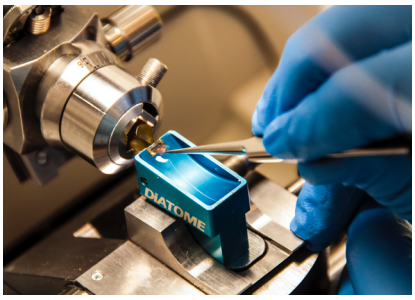


Fig. 3. Es necesario probar cada componente software con precisión.

Vídeo 1. "Pruebas de verdadero o falso"
https://bit.ly/32gcx78

/ 4. Caso práctico 1: "Uso de valores límites"

Planteamiento: En nuestro equipo es necesario realizar una comprobación del software que estamos implementando actualmente, en el cual, permitimos una entrada de valores que admite diferentes tipos de datos. Entre ellos, admitimos los valores numéricos tipo entero para la introducción de los años de nacimiento que se almacenarán en una base de datos.

Nudo: ¿Qué tipo de pruebas utilizarías realizarías para validar que las fechas se introducen correctamente?

Desenlace: Según hemos visto en el tema, existen diferentes tipos de pruebas que permiten validar si los valores introducidos por el usuario son válidos. A la hora de validar si las fechas son correctas en nuestro sistema, nos cabría la duda si tenemos que probar todas las posibles fechas hasta la actualidad. Aunque existen diferentes enfoques para poder realizar la comprobación de los valores de entrada, quizás el más acertado es utilizar los valores límites junto con las clases de equivalencia. De esta forma, no sería necesario comprobar todos los valores enteros de cuatro dígitos, sino que simplemente tendríamos que comprobar aquellos dentro de un rango.

Podríamos fijar los valores límites para personas de hasta 150 años y otro valor límite la fecha actual. De esta forma, el sistema podría aceptar fechas de hasta 150 años atrás.

Para poder determinar si existe una clase de equivalencia cabría preguntarse si existe alguna diferencia entre las fechas que están en ese rango de valores. Si no existe ninguna diferencia podríamos decir que existe una clase de equivalencia entre cada una de ellas.

Por lo tanto, en el rango establecido solo tendríamos que utilizar una fecha determinada más los valores límites para probar el sistema.

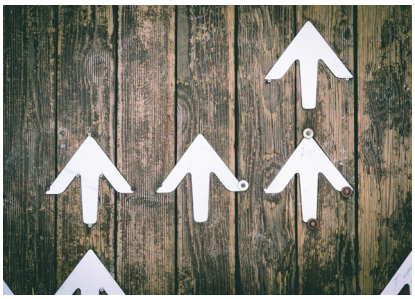


Fig. 4. Deben comprobarse los resultados de las pruebas.

/ 5. Automatización de pruebas

En la actualidad existen diferentes alternativas para poder realizar las pruebas del código de una aplicación. Normalmente, utilizamos ***frameworks* que permiten automatizar la realización y ejecución de las pruebas.** Por ejemplo, existen herramientas que generan casos de prueba para cubrir gran parte del código. A la hora de automatizar una prueba es necesario que la herramienta que utilicemos sea capaz de detectar correctamente los errores y poder documentarlos adecuadamente.

En la actualidad, uno de los campos en los que se desarrollan más aplicaciones es en la computación móvil. Por esta razón surgen determinadas **herramientas que permiten probar fácilmente las aplicaciones que realicemos.** Además, si tenemos en cuenta que existen diferentes plataformas y diferentes enfoques para realizar las aplicaciones móviles, el número de herramientas para la relación de pruebas en este contexto aumenta significativamente.

Tradicionalmente, **la automatización de pruebas se puede dividir en dos grandes grupos:**

A. **Pruebas de código.** Se trata de una prueba que **mide si la funcionalidad que hemos implementado se adapta a los requisitos** teniendo en cuenta diferentes módulos, librerías y clases que intervienen en la ejecución del código. Normalmente, estas pruebas pueden ser automatizadas de una manera sencilla.

B. **Pruebas de interfaz gráfica.** Se trata de pruebas que se centran en **simular la entrada de datos mediante el teclado y eventos de ratón.** Simular la interfaz gráfica y la interacción humana puede ser realmente algo complejo. Por esta razón, se suele utilizar la interacción humana para poder realizar este tipo de pruebas.



Fig. 5. Las pruebas pueden automatizarse.

Existen diferentes tipos de **herramientas para poder lanzar las pruebas de manera automática.** Una de las herramientas más representativas para la automatización de pruebas es **Selenium**. Con esta herramienta *software* se puede probar funcionalmente el producto generado. Permite grabar, editar y depurar los casos de uso y correspondientes casos de prueba.

Audio 2. "Proceso y documentación de pruebas"
http://bit.ly/3no6BD3

/ 6. Calidad del software

La calidad del *software* está **relacionada directamente con las pruebas que realicemos.** Normalmente, **al sacar un producto al mercado** es necesario realizar diferentes tipos de pruebas para asegurar que cumple con algunos **estándares de calidad**. En el **desarrollo *software*** es necesario realizar ciertas validaciones para poder comprobar que no existen errores que puedan poner en peligro la calidad del producto.

En un mundo en el que constantemente salen nuevas aplicaciones, la experiencia de usuario es cada vez más importante, de tal forma que si el usuario no está satisfecho con el producto borrará o dejará de usar la aplicación y perderemos un posible cliente.

Con la evolución de los lenguajes de programación es posible establecer diferentes medidas de control en el código que utilizamos.

En la actualidad, los lenguajes de programación incluyen tratamiento de excepciones para poder evitar que la aplicación aborte de manera inesperada.

Uno de los controles que podemos introducir en el *software* son **las validaciones**. A través de éstas es posible realizar determinados controles como, por ejemplo, comprobar los límites de los vectores, comprobar los archivos utilizados, etc. También es recomendable utilizar las ventajas que nos ofrecen los editores para comprobar los posibles errores que podamos tener en el código.

Existen diferentes **medidas y normas de calidad** cuyo objetivo es poder **garantizar que el *software* cumple con unos ciertos criterios objetivos.** Dependiendo del contexto del desarrollo software se pueden aplicar medidas de **auditoría** al *software* y el proceso de desarrollo. Podemos diferenciar diferentes **normas de calidad ISO** que van dirigidas a garantizar diferentes aspectos de la calidad del *software*. Independientemente de la calidad del código, hay que medir también la calidad del producto final que se está generando, pues en muchas ocasiones la calidad del producto también implica la calidad del proceso en el que se ha desarrollado.

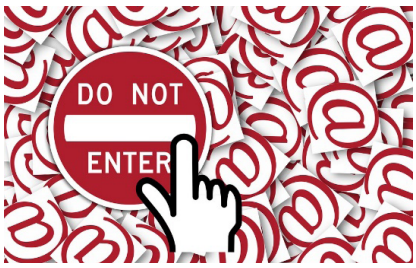


Fig. 6. La ausencia de vulnerabilidades es sinónimo de calidad del software.

Vídeo 2. "Comprobación de la calidad del software2
https://bit.ly/2ZxZFHy

/ 7. Caso práctico 2: "Validación de datos"

Planteamiento: Tras el desarrollo de una aplicación para la compra y venta de vehículos es necesario realizar algunas tareas de validación. Para ello hay que tener en cuenta que dicha aplicación tiene una función que permite medir el número de compras y ventas que se ha realizado por día, mes y año. Para poder sacar el software a producción es necesario poder validar los datos de los cálculos de compra y venta, y fecha.

Nudo: ¿Cómo enfocarías este proceso de pruebas?

Desenlace: Dada la funcionalidad que tiene la aplicación sería necesario realizar algunas de las siguientes pruebas:

- Comprobar el cálculo de gasto de las compras del día.
- Asegurarse que el cálculo del gasto de las compras del mes es correcto.
- Probar que las compras del año dan un total correcto.
- Comprobar que los cálculos de ingreso de las compras del día, del mes y del año son correctos.

Además, dado que estamos trabajando con fechas sería necesario poder probar valores extremos en casos particulares. Por ejemplo, deberíamos revisar qué pasa con los meses que tienen 28 o 31 días. También sería necesario comprobar qué sucede con los años bisiestos. Como podemos observar, existen diferentes aspectos que tendríamos que probar en nuestra aplicación antes de sacarla a producción. A priori, las pruebas pueden parecer relativamente sencillas, pero cuando hacemos foco en ellas vemos que realmente pueden salir muchas pruebas para una aplicación relativamente pequeña. No es de extrañar que las pruebas puedan requerir mucho tiempo, pues requieren que creemos los vectores de prueba y realicemos la ejecución del código con dichos valores.

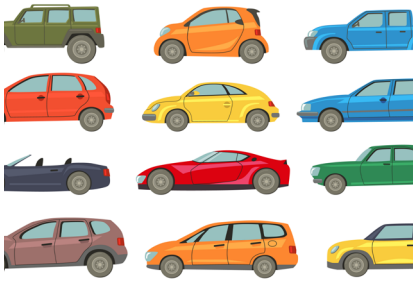


Fig. 7. App para la compra y venta de vehículos.

Una vez que tengamos el resultado habrá que crear la documentación y, si es posible, automatizar el proceso de prueba para lanzarlo en el futuro de manera sistemática. Si somos capaces de hacerlo de esta forma podemos tener sistemas que automáticamente descarguen el código, lo compilen y lancen las pruebas correspondientes a una hora determinada.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema hemos presentado diferentes tipos de pruebas para poder realizar la comprobación funcional y lógica del código que hayamos generado. Hemos podido ver que existen pruebas de **cobertura, pruebas de casos límites y clases de equivalencia** que permiten reducir en gran medida el número de pruebas que tenemos que realizar en nuestra aplicación.

A través de las pruebas podemos garantizar determinados parámetros de calidad de un producto, y también del *software*. Normalmente, las pruebas requieren gran cantidad de tiempo y por ello hemos presentado los conceptos necesarios para poder realizarlas automáticamente.

Existen **diferentes herramientas** para poder **automatizar las pruebas**, pudiendo aumentar la productividad de nuestro equipo al tener que realizar menos tareas y dedicar el esfuerzo a implementar nuevas funcionalidades. También nos hemos centrado en los aspectos de calidad que pueden afectar al desarrollo y comercialización de una aplicación.

Gracias a los **estándares de calidad** podemos asegurar que una aplicación cumple con determinados criterios, para lo cual, es imprescindible la utilización de diferentes tipos pruebas.

Resolución del caso práctico inicial

Como hemos visto en este tema, existen diferentes normas de calidad que permiten garantizar la calidad del *software* que desarrollamos. En general, existen múltiples normas de calidad que pueden afectar no solo al *software,* sino a cualquier producto. El principio básico de estas normas es que deben cumplirse en todo momento, en caso contrario, estarían afectando a la calidad del producto. Tal y como nos comentan en el caso práctico, nos estamos saltando nuestro propio procedimiento de calidad. Esto suele pasar en muchas empresas en las que se define un procedimiento y en ocasiones se incumple.

Es responsabilidad de la empresa volver a asegurar la norma de calidad y seguir el procedimiento establecido previamente. En caso contrario, cuando se enfrente a la correspondiente auditoria, puede perderse el certificado de la norma ISO.



Fig. 8. La calidad es algo que se ha de certificar.

/ 9. Bibliografía

Laporte, C. & April, A. (2018). *Software quality assurance. Hoboken*, NJ: Wiley-IEEE Computer Society, Inc.

Goericke, S. (2019). *The Future of Software Quality Assurance*. Cham: Springer.